

**LAPORAN PRAKTIKUM
PEMROGRAMAN MOBILE
MODUL 3**



BUILD A SCROLLABLE LIST

Oleh:

Muhammad Dzul Fathi Ahyan NIM. 2410817210011

**PROGRAM STUDI TEKNOLOGI INFORMASI
FAKULTAS TEKNIK
UNIVERSITAS LAMBUNG MANGKURAT
APRIL 2026**

LEMBAR PENGESAHAN
LAPORAN PRAKTIKUM PEMROGRAMAN MOBILE
MODUL 3

Laporan Praktikum Pemrograman Mobile Modul 3: Build a Scrollable List ini disusun sebagai syarat lulus mata kuliah Praktikum Pemrograman Mobile. Laporan Praktikum ini dikerjakan oleh:

Nama Praktikan : Muhammad Dzul Fathi Ahyan

NIM : 2410817310009

Menyetujui,
Asisten Praktikum

Mengetahui,
Dosen Penanggung Jawab Praktikum

Raymond Hariyono
NIM. 2310817210007

Erika Maulidiya, S.Kom., M.Kom.
NIP. 200006192025062016

DAFTAR ISI

LEMBAR PENGESAHAN.....	2
DAFTAR ISI	3
DAFTAR GAMBAR.....	4
DAFTAR TABEL	5
SOAL 1.....	6
A. Source Code	8
Source Code XML	8
Source Code Compose	21
B. Output Program.....	31
C. Pembahasan.....	35
Pembahasan XML	35
Pembahasan Compose.....	50
SOAL 2.....	60
REFERENSI.....	62
TAUTAN GIT	63

DAFTAR GAMBAR

Gambar 1. Contoh UI List	7
Gambar 2. Contoh UI Detail.....	7
Gambar 3. Tampilan Awal Aplikasi XML (Vertikal)	31
Gambar 4. Tampilan Halaman Detail Film XML (Vertikal).....	32
Gambar 5. Tampilan Aplikasi FilmApp dengan XML (Horizontal)	32
Gambar 6. Tampilan Halaman Detail Film Compose (Horizontal)	32
Gambar 7. Tampilan Awal Aplikasi FilmApp Compose (Vertikal).....	33
Gambar 8. Tampilan Halaman Detail Film Compose (Vertikal)	33
Gambar 9. Tampilan Aplikasi FilmApp dengan Compose (Horizontal)	34
Gambar 10. Tampilan Halaman Detail Film Compose (Horizontal)	34

DAFTAR TABEL

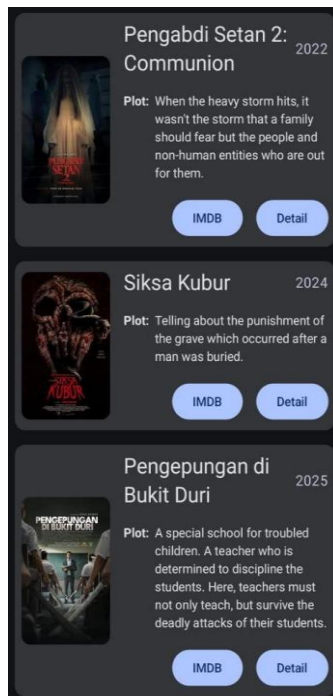
Tabel 1. Source Code FirendData Jawaban Soal 1 XML	8
Tabel 2 Source Code Friend Jawaban Soal 1 XML	9
Tabel 3 Source Code Adapter.kt Jawaban Soal 1 XML	9
Tabel 4 Source Code DetailFragment Jawaban Soal 1 XML	11
Tabel 5 Source Code FriendListFragment Jawaban Soal 1 XML	12
Tabel 6 Source Code MainActivity Jawaban Soal 1 XML	14
Tabel 7 Source Code fragment_detail.xml Jawaban Soal 1 XML	15
Tabel 8 Source Code fragment_friend_list.xml Jawaban Soal 1 XML	16
Tabel 9 Source Code item_friend.xml Jawaban Soal 1 XML	17
Tabel 10 Source Code nav_graph.xml Jawaban Soal 1 XML	20
Tabel 11. Source Code MainActivity.kt Jawaban Soal 1 Compose	21
Tabel 12 Source Code Friend Data Jawaban Soal 1 Compose	22
Tabel 13 Source Code Friend Jawaban Soal 1 Compose	24
Tabel 14 Source Code DetailScreen.kt Jawaban Soal 1 Compose	24
Tabel 15 Source Code FriendItem.kt Jawaban Soal 1 Compose	26
Tabel 16 Souce Code FriendListScreen.kt Jawaban Soal 1 Compose	29

SOAL 1

Buatlah sebuah aplikasi Android menggunakan XML dan Jetpack Compose yang dapat menampilkan list dengan ketentuan berikut:

1. List menggunakan fungsi RecyclerView (XML) dan LazyColumn (Compose)
2. List paling sedikit menampilkan 5 item. Tema item yang ingin ditampilkan bebas
3. Item pada list menampilkan teks dan gambar sesuai dengan contoh di bawah
4. Terdapat 2 button dalam list, dengan fungsi berikut:
5. Button pertama menggunakan intent eksplisit untuk membuka aplikasi atau browser lain
6. Button kedua menggunakan Navigation component untuk membuka laman detail item
7. Sudut item pada list dan gambar di dalam list melengkung atau rounded corner menggunakan Radius
8. Saat orientasi perangkat berubah/dirota, baik ke portrait maupun landscape, aplikasi responsif dan dapat menunjukkan list dengan baik. Data di dalam list tidak boleh hilang
9. Aplikasi menggunakan arsitektur *single activity* (satu activity memiliki beberapa fragment)
10. Aplikasi berbasis XML harus menggunakan ViewBinding.

UI item list harus berisi 1 gambar, 2 button (intent eksplisit dan navigasi), dan 2 baris teks dan setiap baris memiliki 2 teks yang berbeda. Diusahakan agar desain UI item list menyerupai UI berikut



Gambar 1. Contoh UI List

Desain UI laman detail bebas, tetapi diusahakan untuk mengikuti kaidah desain Material Design dan data item ditampilkan penuh di laman detail seperti contoh berikut



Gambar 2. Contoh UI Detail

A. Source Code

Source Code XML

FriendData

Tabel 1. Source Code FirendData Jawaban Soal 1 XML

1	package com.example.modul3xml.data
2	
3	import com.example.modul3xml.R
4	import com.example.modul3xml.model.Friend
5	
6	object FriendData {
7	val friends = listOf(
8	Friend(1, "Muhammad Lv Gunawuy", "32 Agustus 6969", "Foto lv ini diambil ketika dia ingin bercita cita menguasai dunia dan bertekad bahwa dia akan menjadi heavenly demon perkap yang tidak terkalahkan dan juga akan mmenentukan penerus heavenly demon perkap selanjutnya di bawah kepemimpinan dia", R.drawable.lv, "https://instagram.com/"),
9	Friend(2, "Fatih Bari Muar", "29 Masehi 3802", "Fatih merupakan seorang puitisi yang sangat tidak terkenal dan tidak akan terkenal kerna puitisi dia hanya dapat digunakan dalam sekali pakai, foto ini diambil ketika dia sedang ingin buang hajat dan ingin mencari insiparsi untuk puisi yang akan dia buat selanjutnya", R.drawable.fatih, "https://instagram.com/"),
10	Friend(3, "Meluyyyyyyy", "32 March 12738", "Foto ini ketika Meluy sedang bermain sebua aplikasi seperti ome tv dan sedang mencari para cwk cwk yang akan digoda oleh meluy tetapi dia malah bertemu seorang pria dan dihina hina oleh pria tersebut sebagai chain smoker", R.drawable.melon, "https://instagram.com/"),
11	Friend(4, "neRvers.", "43 April 21387", "Ini adalah foto seorang neRvers. ketika sedang lock in untuk melakukan ibadah, tapi disini dia sedang lock in karena setelah selesai ibadah dia ingin mencucuk pentol amang amang di muka masjid", R.drawable.nafis, "https://instagram.com/"),
12	Friend(5, "Rakha x Police", "48 Janus 2317", "Foto ini adalah foto seorang yang ingin bercita cita menjadi seorang police yaitu rakha TI 24 FT ULM WAKAHIM HMTI FT ULM yang diman foto ini dia sedang melakukan wawancara kepada calon calon anggota HMTI FT ULM dengan dia berniat untuk menggagalkan semua orang", R.drawable.rakha, "https://instagram.com/"))

13	)
14	}

Friend

Tabel 2 Source Code Friend Jawaban Soal 1 XML

1	package com.example.modul3xml.model
2	
3	import androidx.annotation.DrawableRes
4	
5	data class Friend(
6	val id: Int,
7	val name: String,
8	val date: String,
9	val feature: String,
10	@DrawableRes val photoResId: Int,
11	val instagramUrl: String
12	)

Adapter.kt

Tabel 3 Source Code Adapter.kt Jawaban Soal 1 XML

1	package com.example.modul3xml.ui
2	
3	import android.content.Intent
4	import android.net.Uri
5	import android.view.LayoutInflater
6	import android.view.ViewGroup
7	import androidx.recyclerview.widget.RecyclerView
8	import com.bumptech.glide.Glide
9	import
	com.example.modul3xml.databinding.ItemFriendBinding
10	import com.example.modul3xml.model.Friend
11	
12	class FriendAdapter(
13	private val friends: List<Friend>,
14	private val onDetailClick: (Int) -> Unit
15	)
	RecyclerView.Adapter<FriendAdapter.FriendViewHolder>() {
16	
17	// ViewHolder: Kelas yang bertugas memegang dan
	mengelola 1 kotak item_friend.xml
18	inner class FriendViewHolder(private val binding:
	ItemFriendBinding)
	RecyclerView.ViewHolder(binding.root) {

```

19
20         fun bind(friend: Friend) {
21             // Memasukkan teks ke dalam id yang kamu buat
di XML tadi
22             binding.tvName.text = friend.name
23             binding.tvDate.text = friend.date
24             binding.tvFeature.text      =      "Feature:
${friend.feature}"
25
26             // Memasukkan gambar menggunakan library Glide
agar tidak lag
27             Glide.with(itemView.context)
28                 .load(friend.photoResId)
29                 .into(binding.ivPhoto)
30
31             // Mengatur tombol Instagram (Intent
Eksplisit)
32             binding.btnInstagram.setOnClickListener {
33                 val intent = Intent(Intent.ACTION_VIEW,
Uri.parse(friend.instagramUrl))
34                 itemView.context.startActivity(intent)
35             }
36
37             // Mengatur tombol Detail (Memicu navigasi
dengan membawa ID)
38             binding.btnDetail.setOnClickListener {
39                 onDetailClick(friend.id)
40             }
41         }
42     }
43
44     // Dipanggil saat RecyclerView butuh mencetak kotak
kosong baru
45     override fun onCreateViewHolder(parent: ViewGroup,
viewType: Int): FriendViewHolder {
46         val binding =
ItemFriendBinding.inflate(LayoutInflater.from(parent.con
text), parent, false)
47         return FriendViewHolder(binding)
48     }
49
50     // Memberitahu RecyclerView berapa banyak total data
temanmu
51     override fun getItemCount(): Int = friends.size
52
53     // Mengisi kotak kosong tersebut dengan data
berdasarkan urutannya

```

54	override fun onBindViewHolder(holder: FriendViewHolder, position: Int) {
55	holder.bind(friends[position])
56	}
57	}

DetailFragment

Tabel 4 Source Code DetailFragment Jawaban Soal 1 XML

1	package com.example.modul3xml.ui
2	
3	import android.os.Bundle
4	import android.view.LayoutInflater
5	import android.view.View
6	import android.view.ViewGroup
7	import androidx.fragment.app.Fragment
8	import com.bumptech.glide.Glide
9	import com.example.modul3xml.data.FriendData
10	import
	com.example.modul3xml.databinding.FragmentDetailBinding
11	
12	class DetailFragment : Fragment() {
13	
14	private var _binding: FragmentDetailBinding? = null
15	private val binding get() = _binding!!
16	
17	override fun onCreateView(
18	inflater: LayoutInflater, container: ViewGroup?,
19	savedInstanceState: Bundle?
20	): View {
21	_binding =
	FragmentDetailBinding.inflate(inflater, container, false)
22	return binding.root
23	}
24	
25	override fun onViewCreated(view: View,
	savedInstanceState: Bundle?) {
26	super.onViewCreated(view, savedInstanceState)
27	
28	// 1. Menangkap ID teman yang dikirim dari layar
	list
29	val friendId = arguments?.getInt("friendId") ?: 0
30	
31	// 2. Mencari data teman di FriendData berdasarkan
	ID tersebut

```

32         val friend = FriendData.friends.find { it.id ==
friendId }
33
34         if (friend != null) {
35             // 3. Jika ketemu masukkan datanya ke komponen
visual XML
36             binding.tvDetailName.text = friend.name
37             binding.tvDetailDate.text = friend.date
38             binding.tvDetailFeature.text = friend.feature
39
40             // 4. Memuat gambar besar menggunakan Glide
41             Glide.with(this)
42                 .load(friend.photoResId)
43                 .into(binding.ivDetailPhoto)
44         }
45     }
46
47     override fun onDestroyView() {
48         super.onDestroyView()
49         _binding = null
50     }
51 }

```

FriendListFragment

Tabel 5 Source Code FriendListFragment Jawaban Soal 1 XML

```

1 package com.example.modul3xml.ui
2
3 import android.os.Bundle
4 import android.view.LayoutInflater
5 import android.view.View
6 import android.view.ViewGroup
7 import androidx.fragment.app.Fragment
8 import androidx.navigation.fragment.findNavController
9 import com.example.modul3xml.R
10 import com.example.modul3xml.data.FriendData
11 import
com.example.modul3xml.databinding.FragmentFriendListBindi
ng
12
13 class FriendListFragment : Fragment() {
14
15     // Setup ViewBinding khusus untuk Fragment
16     private var _binding: FragmentFriendListBinding? =
null
17     private val binding get() = _binding!!

```

```

18
19 // Fungsi ini bertugas "mencetak" tampilan
fragment_friend_list.xml
20 override fun onCreateView(
21     inflater: LayoutInflater, container: ViewGroup?,
22     savedInstanceState: Bundle?
23 ): View {
24     _binding =
FragmentFriendListBinding.inflate(inflater, container,
false)
25     return binding.root
26 }
27
28 // Fungsi ini bertugas mengisi data setelah tampilannya
selesai dicetak
29 override fun onViewCreated(view: View,
savedInstanceState: Bundle?) {
30     super.onViewCreated(view, savedInstanceState)
31
32     // Mengambil gudang data temanmu
33     val friends = FriendData.friends
34
35
36     val featuredAdapter =
FriendAdapter(friends.take(5)) { friendId ->
37         navigateToDetail(friendId)
38     }
39     binding.rvFeaturedFriends.adapter =
featuredAdapter
40
41     // 2. Setup Rak Bawah (All - Ambil semua teman)
42     val allAdapter = FriendAdapter(friends) { friendId
->
43         navigateToDetail(friendId)
44     }
45     binding.rvAllFriends.adapter = allAdapter
46 }
47
48 // Fungsi untuk berpindah ke layar Detail membawa ID
49 private fun navigateToDetail(friendId: Int) {
50     // Bundle adalah "kotak paket" untuk membawa data
antar layar di XML
51     val bundle = Bundle().apply {
52         putInt("friendId", friendId)
53     }
54     // Menyuruh sistem navigasi bergerak ke rute Detail
membawa kotak paket

```

```

55 findNavController().navigate(R.id.action_friendListFragme
56 nt_to_detailFragment, bundle)
57 }
58 // Wajib di Fragment: Menghapus binding saat layar
59 ditutup agar tidak membebani memori
60 override fun onDestroyView() {
61     super.onDestroyView()
62     _binding = null
63 }

```

MainActivity

Tabel 6 Source Code MainActivity Jawaban Soal 1 XML

```

1 package com.example.modul3xml
2
3 import android.os.Bundle
4 import androidx.appcompat.app.AppCompatActivity
5 import
6 com.example.modul3xml.databinding.ActivityMainBinding
7
8 class MainActivity : AppCompatActivity() {
9     private lateinit var binding: ActivityMainBinding
10
11     override fun onCreate(savedInstanceState: Bundle?) {
12         super.onCreate(savedInstanceState)
13
14         // Membaca file activity_main.xml dan
15         menampilkan ke layar binding =
16         ActivityMainBinding.inflate(layoutInflater)
17         setContentView(binding.root)
18     }
19 }

```

Fragment_detail.xml

Tabel 7 Source Code fragment_detail.xml Jawaban Soal 1 XML

```
1 <?xml version="1.0" encoding="utf-8"?>
2 <ScrollView
  xmlns:android="http://schemas.android.com/apk/res/android"
3     xmlns:app="http://schemas.android.com/apk/res-auto"
4     xmlns:tools="http://schemas.android.com/tools"
5     android:layout_width="match_parent"
6     android:layout_height="match_parent"
7     tools:context=".ui.DetailFragment">
8
9     <LinearLayout
10         android:layout_width="match_parent"
11         android:layout_height="wrap_content"
12         android:orientation="vertical"
13         android:padding="16dp">
14
15         <com.google.android.material.imageview.ShapeableImageVie
16             w
17             android:id="@+id/ivDetailPhoto"
18             android:layout_width="match_parent"
19             android:layout_height="350dp"
20             android:scaleType="centerCrop"
21
22             app:shapeAppearanceOverlay="@style/RoundedImage" />
23
24         <TextView
25             android:id="@+id/tvDetailName"
26             android:layout_width="wrap_content"
27             android:layout_height="wrap_content"
28             android:layout_marginTop="16dp"
29             android:textSize="28sp"
30             android:textStyle="bold"
31             tools:text="Nama Teman" />
32
33         <TextView
34             android:id="@+id/tvDetailDate"
35             android:layout_width="wrap_content"
36             android:layout_height="wrap_content"
37             android:layout_marginTop="4dp"
38             android:layout_marginBottom="16dp"
39             android:textSize="16sp"
40             android:textColor="#666666"
41             tools:text="Tanggal" />
```

```

40
41     <TextView
42         android:layout_width="wrap_content"
43         android:layout_height="wrap_content"
44         android:text="Feature:"
45         android:textSize="18sp"
46         android:textStyle="bold" />
47
48     <TextView
49         android:id="@+id/tvDetailFeature"
50         android:layout_width="match_parent"
51         android:layout_height="wrap_content"
52         android:layout_marginTop="4dp"
53         android:textSize="16sp"
54         android:lineSpacingExtra="8sp"
55         tools:text="Deskripsi lengkap teman akan
56 muncul di sini" />
57 </LinearLayout>
58 </ScrollView>

```

Fragment_friend_list.xml

Tabel 8 Source Code fragment_friend_list.xml Jawaban Soal 1 XML

```

1  <?xml version="1.0" encoding="utf-8"?>
2  <androidx.core.widget.NestedScrollView
3      xmlns:android="http://schemas.android.com/apk/res/android"
4      xmlns:app="http://schemas.android.com/apk/res-auto"
5      xmlns:tools="http://schemas.android.com/tools"
6      android:layout_width="match_parent"
7      android:layout_height="match_parent"
8      tools:context=".ui.FriendListFragment">
9      <LinearLayout
10         android:layout_width="match_parent"
11         android:layout_height="wrap_content"
12         android:orientation="vertical"
13         android:paddingBottom="16dp">
14
15         <TextView
16             android:layout_width="wrap_content"
17             android:layout_height="wrap_content"
18             android:layout_margin="16dp"
19             android:text="Featured Friends"
20             android:textSize="20sp"

```


21	android:textStyle="bold" />
22	
23	<androidx.recyclerview.widget.RecyclerView
24	android:id="@+id/rvFeaturedFriends"
25	android:layout_width="match_parent"
26	android:layout_height="wrap_content"
27	android:clipToPadding="false"
28	android:orientation="horizontal"
29	android:paddingHorizontal="8dp"
30	
	app:layoutManager="androidx.recyclerview.widget.LinearLa
	youtManager" />
31	
32	<TextView
33	android:layout_width="wrap_content"
34	android:layout_height="wrap_content"
35	android:layout_margin="16dp"
36	android:text="All Friends"
37	android:textSize="20sp"
38	android:textStyle="bold" />
39	
40	<androidx.recyclerview.widget.RecyclerView
41	android:id="@+id/rvAllFriends"
42	android:layout_width="match_parent"
43	android:layout_height="wrap_content"
44	android:nestedScrollingEnabled="false"
45	
	app:layoutManager="androidx.recyclerview.widget.LinearLa
	youtManager" />
46	
47	</LinearLayout>
48	</androidx.core.widget.NestedScrollView>

Item_friend.xml

Tabel 9 Source Code item_friend.xml Jawaban Soal 1 XML

1	<?xml version="1.0" encoding="utf-8"?>
2	<com.google.android.material.card.MaterialCardView
3	
	xmlns:android="http://schemas.android.com/apk/res/andro
	id"
4	xmlns:app="http://schemas.android.com/apk/res-auto"
5	android:layout_width="match_parent"
6	android:layout_height="wrap_content"
7	android:layout_marginHorizontal="16dp"
8	android:layout_marginVertical="8dp"

```

9      app:cardCornerRadius="16dp"
10     app:cardElevation="2dp">
11
12     <LinearLayout
13         android:layout_width="match_parent"
14         android:layout_height="wrap_content"
15         android:orientation="horizontal"
16         android:padding="16dp"
17         android:gravity="center_vertical">
18
19
20         <com.google.android.material.imageview.ShapeableImageVi
21         ew
22             android:id="@+id/ivPhoto"
23             android:layout_width="100dp"
24             android:layout_height="100dp"
25             android:scaleType="centerCrop"
26
27             app:shapeAppearanceOverlay="@style/RoundedImage" />
28
29         <LinearLayout
30             android:layout_width="0dp"
31             android:layout_height="wrap_content"
32             android:layout_weight="1"
33             android:layout_marginStart="16dp"
34             android:orientation="vertical">
35
36             <LinearLayout
37                 android:layout_width="match_parent"
38                 android:layout_height="wrap_content"
39                 android:orientation="horizontal"
40                 android:gravity="top">
41
42                 <TextView
43                     android:id="@+id/tvName"
44                     android:layout_width="0dp"
45
46                     android:layout_height="wrap_content"
47                     android:layout_weight="1"
48                     android:text="Nama Teman"
49                     android:textSize="16sp"
50                     android:textStyle="bold"
51                     android:maxLines="2"
52                     android:ellipsize="end"/> <TextView
53                     android:id="@+id/tvDate"
54                     android:layout_width="wrap_content"
55                     android:layout_height="wrap_content"

```

```

52         android:text="Tanggal"
53         android:textSize="10sp"
54         android:layout_marginStart="8dp"/>
55     </LinearLayout>
56
57     <TextView
58         android:id="@+id/tvFeature"
59         android:layout_width="match_parent"
60         android:layout_height="wrap_content"
61         android:layout_marginTop="8dp"
62         android:text="Feature:          Deskripsi
teman..."
63         android:textSize="12sp"
64         android:maxLines="2"
65         android:ellipsize="end"/>
66
67     <LinearLayout
68         android:layout_width="match_parent"
69         android:layout_height="wrap_content"
70         android:orientation="horizontal"
71         android:layout_marginTop="12dp">
72
73         <Button
74             android:id="@+id/btnInstagram"
75             android:layout_width="0dp"
76             android:layout_height="wrap_content"
77             android:layout_weight="1"
78             android:text="Instagram"
79             android:textSize="11sp"
80             android:layout_marginEnd="4dp"/>
81
82         <Button
83             android:id="@+id/btnDetail"
84             android:layout_width="0dp"
85             android:layout_height="wrap_content"
86             android:layout_weight="1"
87             android:text="Detail"
88             android:textSize="11sp"
89             android:layout_marginStart="4dp"/>
90     </LinearLayout>
91
92 </LinearLayout>
93 </LinearLayout>
94 </com.google.android.material.card.MaterialCardView>

```

Nav_graph.xml

Tabel 10 Source Code nav_graph.xml Jawaban Soal 1 XML

1	<?xml version="1.0" encoding="utf-8"?>
2	<navigation
	xmlns:android="http://schemas.android.com/apk/res/andro
	id"
3	xmlns:app="http://schemas.android.com/apk/res-auto"
4	xmlns:tools="http://schemas.android.com/tools"
5	android:id="@+id/nav_graph"
6	app:startDestination="@id/friendListFragment">
7	
8	<fragment
9	android:id="@+id/friendListFragment"
10	
	android:name="com.example.modul3xml.ui.FriendListFragme
	nt"
11	android:label="Friend List"
12	tools:layout="@layout/fragment_friend_list">
13	
14	<action
15	
	android:id="@+id/action_friendListFragment_to_detailFra
	gment"
16	app:destination="@id/detailFragment" />
17	</fragment>
18	
19	<fragment
20	android:id="@+id/detailFragment"
21	
	android:name="com.example.modul3xml.ui.DetailFragment"
22	android:label="Detail Friend"
23	tools:layout="@layout/fragment_detail">
24	
25	<argument
26	android:name="friendId"
27	app:argType="integer"
28	android:defaultValue="0" />
29	</fragment>
30	
31	</navigation>

Source Code Compose

MainActivity.kt

Tabel 11. Source Code MainActivity.kt Jawaban Soal 1 Compose

```
1 package com.example.modul3compose
2
3 import android.os.Bundle
4 import androidx.activity.ComponentActivity
5 import androidx.activity.compose.setContent
6 import androidx.activity.enableEdgeToEdge
7 import androidx.compose.foundation.layout.fillMaxSize
8 import androidx.compose.foundation.layout.padding
9 import androidx.compose.material3.Scaffold
10 import androidx.compose.ui.Modifier
11 import androidx.navigation.NavType
12 import androidx.navigation.compose.NavHost
13 import androidx.navigation.compose.composable
14 import androidx.navigation.compose.rememberNavController
15 import androidx.navigation.navArgument
16 import com.example.modul3compose.ui.DetailScreen
17 import com.example.modul3compose.ui.FriendListScreen
18 import
com.example.modul3compose.ui.theme.Modul3ComposeTheme
19
20 class MainActivity : ComponentActivity() {
21     override fun onCreate(savedInstanceState: Bundle?) {
22         super.onCreate(savedInstanceState)
23         enableEdgeToEdge()
24         setContent {
25             Modul3ComposeTheme {
26                 Scaffold(modifier
Modifier.fillMaxSize()) { innerPadding ->
27
28                 val navController =
rememberNavController()
29
30                 NavHost(
31                     navController = navController,
32                     startDestination
"list_screen",
33                     modifier
Modifier.padding(innerPadding)
34                 ) {
35
36                     composable("list_screen") {
37                         FriendListScreen(
```

```

38             onNavigateToDetail = {
39                 friendId ->
40                     navController.navigate("detail_screen/${friendId}")
41                     }
42             }
43
44             composable(
45                 route =
46                 "detail_screen/{friendId}",
47                 arguments =
48                 listOf(navArgument("friendId") { type = NavType.IntType
49                     }) { backStackEntry ->
50                     val id =
51                     backStackEntry.arguments?.getInt("friendId") ?: 0
52                     DetailScreen(friendId = id)
53                 }
54             }
55         }
56     }
57 }
58 }

```

FriendData

Tabel 12 Source Code Friend Data Jawaban Soal 1 Compose

```

1 package com.example.modul3compose.data
2
3 import com.example.modul3compose.R
4 import com.example.modul3compose.model.Friend
5
6 object FriendData {
7     val friends = listOf(
8         Friend(
9             id = 1,
10            name = "Muhammad Lv Gunawuy",
11            date = "32 Agustus 6969",
12            feature = "Foto lv ini diambil ketika dia
            ingin bercita cita menguasai dunia dan bertekad bahwa dia
            akan menjadi heavenly demon perkap yang tidak terkalahkan

```

	dan juga akan mmenentukan penerus heavenly demon perkap selanjutnya di bawah kepemimpinan dia",
13	photoResId = R.drawable.lv,
14	instagramUrl = "https://instagram.com/"
15	),
16	Friend(
17	id = 2,
18	name = "Fatih Bari Muar",
19	date = "29 Masehi 3802",
20	feature = "Fatih merupakan seorang puitisi yang sangat tidak terkenal dan tidak akan terkenal kerna puitisi dia hanya dapat digunakan dalam sekali pakai, foto ini diambil ketika dia sedang ingin buang hajat dan ingin mencari insiparsi untuk puisi yang akan dia buat selanjutnya",
21	photoResId = R.drawable.fatih,
22	instagramUrl = "https://instagram.com/"
23	),
24	Friend(
25	id = 3,
26	name = "Meluyyyyyyy",
27	date = "32 March 12738",
28	feature = "Foto ini ketika Meluy sedang bermain sebua aplikasi seperti ome tv dan sedang mencari para cwk cwk yang akan digoda oleh meluy tetapi dia malah bertemu seorang pria dan dihina hina oleh pria tersebut sebagai chain smoker",
29	photoResId = R.drawable.melon,
30	instagramUrl = "https://instagram.com/"
31	),
32	Friend(
33	id = 4,
34	name = "neRvers.",
35	date = "43 April 21387",
36	feature = "Ini adalah foto seorang neRvers. ketika sedang lock in untuk melakukan ibadah, tapi disini dia sedang lock in karena setelah selesai ibadah dia ingin mencucuk pentol amang amang di muka masjid",
37	photoResId = R.drawable.nafis,
38	instagramUrl = "https://instagram.com/"
39	),
40	Friend(
41	id = 5,
42	name = "Rakha x Police",
43	date = "48 Janus 2317",
44	feature = "Foto ini adalah foto seorang yang ingin bercita cita menjadi seorang police yaitu rakha TI

	24 FT ULM WAKAHIM HMTI FT ULM yang diman foto ini dia sedang melakukan wawancara kepada calon calon anggota HMTI FT ULM dengan dia berniat untuk menggagalkan semua orang",
45	photoResId = R.drawable.rakha,
46	instagramUrl = "https://instagram.com/"
47	)
48	)
49	}

Friend

Tabel 13 Source Code Friend Jawaban Soal 1 Compose

1	package com.example.modul3compose.model
2	
3	import androidx.annotation.DrawableRes
4	
5	data class Friend(
6	val id: Int,
7	val name: String,
8	val date: String,
9	val feature: String,
10	@DrawableRes val photoResId: Int,
11	val instagramUrl: String
12	)

DetailScreen.kt

Tabel 14 Source Code DetailScreen.kt Jawaban Soal 1 Compose

1	package com.example.modul3compose.ui
2	
3	import androidx.compose.foundation.Image
4	import androidx.compose.foundation.layout.*
5	import androidx.compose.foundation.rememberScrollState
6	import
	androidx.compose.foundation.shape.RoundedCornerShape
7	import androidx.compose.foundation.verticalScroll
8	import androidx.compose.material3.*
9	import androidx.compose.runtime.Composable
10	import androidx.compose.ui.Modifier
11	import androidx.compose.ui.draw.clip


```

12 import androidx.compose.ui.layout.ContentScale
13 import androidx.compose.ui.res.painterResource
14 import androidx.compose.ui.text.font.FontWeight
15 import androidx.compose.ui.unit.dp
16 import androidx.compose.ui.unit.sp
17 import com.example.modul3compose.data.FriendData
18
19 @Composable
20 fun DetailScreen(friendId: Int, modifier: Modifier =
Modifier) {
21     // Mencari data teman di FriendData berdasarkan ID
    yang dikirim
22     val friend = FriendData.friends.find { it.id ==
friendId }
23
24     if (friend != null) {
25         Column(
26             modifier = modifier
27                 .fillMaxSize()
28                 .padding(16.dp)
29                 .verticalScroll(rememberScrollState())
30         ) {
31             Image(
32                 painter = painterResource(id =
friend.photoResId),
33                 contentDescription = friend.name,
34                 contentScale = ContentScale.Crop,
35                 modifier = Modifier
36                     .fillMaxWidth()
37                     .height(350.dp)
38                     .clip(RoundedCornerShape(16.dp))
39             )
40
41             Spacer(modifier = Modifier.height(16.dp))
42
43             // Teks Judul
44             Text(
45                 text = friend.name,
46                 fontSize = 28.sp,
47                 fontWeight = FontWeight.Bold,
48                 lineHeight = 32.sp
49             )
50
51             //Tanggal / Keterangan Waktu
52             Text(
53                 text = friend.date,
54                 fontSize = 16.sp,

```

55	color	=
56	MaterialTheme.colorScheme.onSurfaceVariant,	
	modifier = Modifier.padding(top = 4.dp,	
	bottom = 16.dp)	
57	)	
58		
59	// Label Deskripsi	
60	Text(	
61	text = "Feature:",	
62	fontSize = 18.sp,	
63	fontWeight = FontWeight.Bold	
64	)	
65		
66	Spacer(modifier = Modifier.height(4.dp))	
67		
68	// Isi Deskripsi Lengkap	
69	Text(	
70	text = friend.feature,	
71	fontSize = 16.sp,	
72	lineHeight = 24.sp	
73	)	
74	}	
75	} else {	
76	// if tidak ditemukan	
77	Box(modifier = Modifier.fillMaxSize(),	
	contentAlignment = androidx.compose.ui.Alignment.Center)	
	{	
78	Text("Data teman tidak ditemukan")	
79	}	
80	}	
81	}	

FriendItem.kt

Tabel 15 Source Code FriendItem.kt Jawaban Soal 1 Compose

1	package com.example.modul3compose.ui
2	
3	import androidx.compose.foundation.Image
4	import androidx.compose.foundation.layout.*
5	import
	androidx.compose.foundation.shape.RoundedCornerShape
6	import androidx.compose.material3.*
7	import androidx.compose.runtime.Composable
8	import androidx.compose.ui.Modifier
9	import androidx.compose.ui.draw.clip
10	import androidx.compose.ui.layout.ContentScale

```

11 import androidx.compose.ui.platform.LocalContext
12 import androidx.compose.ui.res.painterResource
13 import androidx.compose.ui.text.font.FontWeight
14 import androidx.compose.ui.text.style.TextOverflow
15 import androidx.compose.ui.unit.dp
16 import androidx.compose.ui.unit.sp
17 import com.example.modul3compose.model.Friend
18
19 @Composable
20 fun FriendItem(
21     friend: Friend,
22     onDetailClick: () -> Unit,
23     modifier: Modifier = Modifier
24 ) {
25     val context = LocalContext.current
26
27     Card(
28         shape = RoundedCornerShape(16.dp),
29         colors = CardDefaults.cardColors(containerColor
= MaterialTheme.colorScheme.surfaceVariant),
30         modifier = modifier
31             .padding(horizontal = 8.dp, vertical = 8.dp)
32     ) {
33         Row(
34             modifier = Modifier.padding(16.dp),
35             horizontalArrangement
=
Arrangement.spacedBy(16.dp)
36         ) {
37             Image(
38                 painter
= painterResource(id
=
friend.photoResId),
39                 contentDescription = friend.name,
40                 contentScale = ContentScale.Crop,
41                 modifier = Modifier
42                     .size(100.dp)
43                     .clip(RoundedCornerShape(12.dp))
44             )
45
46             Column(
47                 modifier = Modifier.weight(1f)
48             ) {
49
50                 Row(
51                     modifier = Modifier.fillMaxWidth(),
52                     horizontalArrangement
=
Arrangement.SpaceBetween
53                 ) {

```

```

54
55         Text(
56             text = friend.name,
57             fontWeight = FontWeight.Bold,
58             fontSize = 16.sp,
59             maxLines = 1,
60             overflow =
TextOverflow.Ellipsis,
61             modifier = Modifier.weight(1f)
62         )
63
64         Text(
65             text = friend.date,
66             fontSize = 10.sp,
67             modifier =
Modifier.padding(start = 8.dp)
68         )
69
70     }
71
72     Spacer(modifier = Modifier.height(8.dp))
73
74     Text(
75         text = friend.feature,
76         fontSize = 12.sp,
77         maxLines = 4,
78         overflow = TextOverflow.Ellipsis
79     )
80
81     Spacer(modifier =
Modifier.height(12.dp))
82
83     Row(
84         modifier = Modifier.fillMaxWidth(),
85         horizontalArrangement =
Arrangement.spacedBy(8.dp)
86     ) {
87
88         Button(
89             onClick = { },
90             contentPadding =
PaddingValues(horizontal = 4.dp, vertical = 8.dp),
91             modifier = Modifier.weight(1f),
92             colors =
ButtonDefaults.buttonColors()
93         ) {

```

```

94         Text("Instagram", fontSize = 11.sp,
maxLines = 1, overflow = TextOverflow.Ellipsis)
95     }
96
97     Button(
98         onClick = onDetailClick,
99         contentPadding
=
PaddingValues(horizontal = 4.dp, vertical = 8.dp),
100         modifier = Modifier.weight(1f),
101         colors
=
ButtonDefaults.buttonColors()
102     ) {
103         Text("Detail", fontSize = 11.sp,
maxLines = 1, overflow = TextOverflow.Ellipsis)
104     }
105 }
106 }
107 }
108 }
109 }

```

FriendListScreen.kt

Tabel 16 Source Code FriendListScreen.kt Jawaban Soal 1 Compose

```

1 package com.example.modul3compose.ui
2
3 import androidx.compose.foundation.layout.*
4 import androidx.compose.foundation.lazy.LazyColumn
5 import androidx.compose.foundation.lazy.LazyRow
6 import androidx.compose.foundation.lazy.items
7 import androidx.compose.material3.Text
8 import androidx.compose.runtime.Composable
9 import androidx.compose.ui.Modifier
10 import androidx.compose.ui.text.font.FontWeight
11 import androidx.compose.ui.unit.dp
12 import androidx.compose.ui.unit.sp
13 import com.example.modul3compose.data.FriendData
14
15 @Composable
16 fun FriendListScreen(
17     onNavigateToDetail: (Int) -> Unit,
18     modifier: Modifier = Modifier
19 ) {
20
21     val friends = FriendData.friends
22

```

```

23     LazyColumn(
24         modifier = modifier
25             .fillMaxSize(),
26     ) {
27         item {
28             Text(
29                 text = "Featured Friends",
30                 fontWeight = FontWeight.Bold,
31                 fontSize = 20.sp,
32                 modifier = Modifier.padding(start =
16.dp, top = 16.dp, bottom = 8.dp)
33             )
34         }
35
36         item {
37             LazyRow(
38                 contentPadding
39                 PaddingValues(horizontal = 16.dp),
40                 horizontalArrangement
41                 Arrangement.spacedBy(16.dp)
42             ) {
43                 items(friends.take(5)) { friend ->
44                     Box
45                     (modifier
46                     Modifier.width(350.dp)) {
47                         FriendItem(
48                             friend = friend,
49                             onDetailClick
50                             =
51                             {
52                                 onNavigateToDetail(friend.id) }
53                             )
54                         }
55                     }
56                 }
57             }
58
59             item {
60                 Text(
61                     text = "All Friends",
62                     fontWeight = FontWeight.Bold,
63                     fontSize = 20.sp,
64                     modifier = Modifier.padding(start =
16.dp, top = 24.dp, bottom = 8.dp)
65                 )
66             }
67
68             items(friends) { friend ->

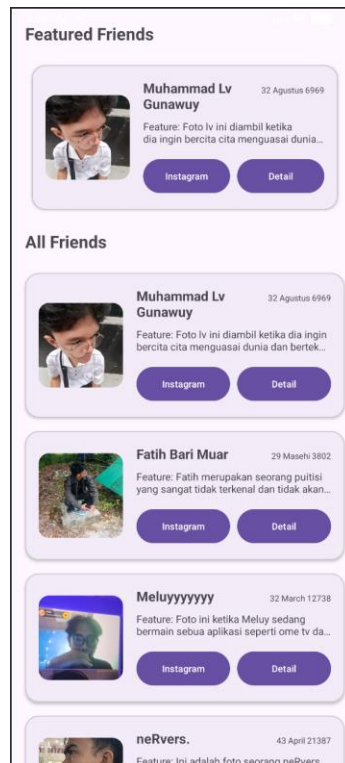
```

```

64         FriendItem(
65             friend = friend,
66             onDetailClick = {
onNavigateToDetail(friend.id) }
67         )
68     }
69
70 }
71 }

```

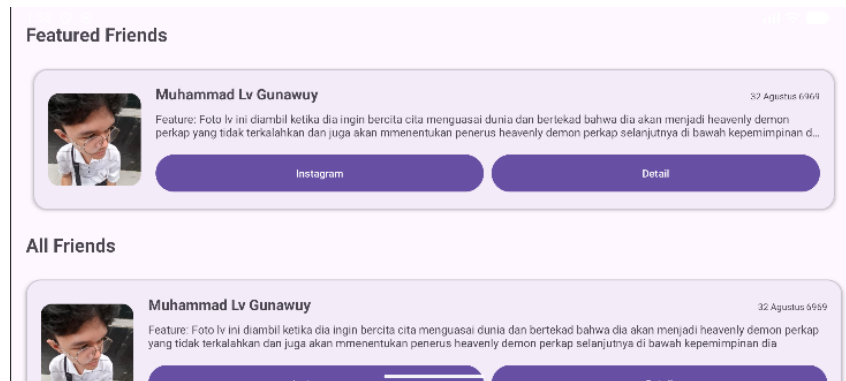
B. Output Program



Gambar 3. Tampilan Awal Aplikasi XML (Vertikal)



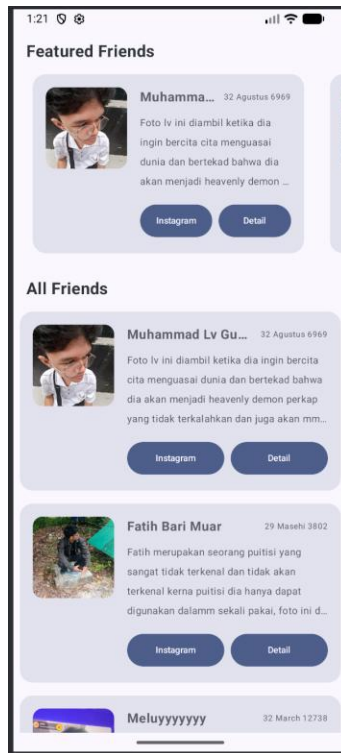
Gambar 4. Tampilan Halaman Detail Film XML (Vertikal)



Gambar 5. Tampilan Aplikasi FilmApp dengan XML (Horizontal)



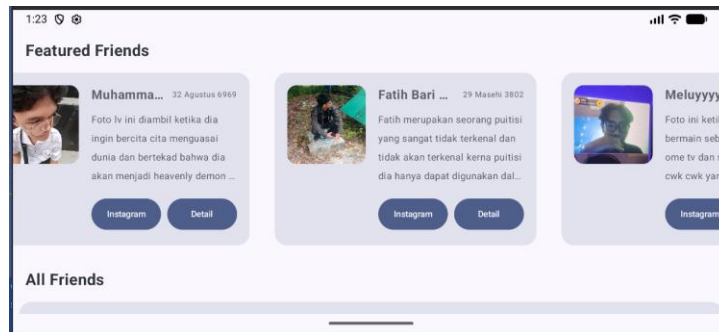
Gambar 6. Tampilan Halaman Detail Film Compose (Horizontal)



Gambar 7. Tampilan Awal Aplikasi FilmApp Compose (Vertikal)



Gambar 8. Tampilan Halaman Detail Film Compose (Vertikal)



Gambar 9. Tampilan Aplikasi FilmApp dengan Compose (Horizontal)



Gambar 10. Tampilan Halaman Detail Film Compose (Horizontal)

C. Pembahasan

Pembahasan XML

FriendData:

Line 1: Deklarasi package `com.example.modul3xml.data`. Ini nandain kalau posisi file ini disimpan khusus di dalam folder data.

Line 3: Blok import (yang posisinya sedang dilipat/di-collapse di Android Studio-mu). Kalau dibuka, isinya memanggil `R` (buat ngambil file gambar dari dalam folder `res/drawable`) dan memanggil cetakan model `Friend` yang sebelumnya udah kamu buat.

Line 6: Deklarasi object `FriendData {}`. Penggunaan kata object di Kotlin (bukan class) berarti kita membuat Singleton. Intinya, gudang data ini cuma diciptakan satu kali di memori HP, jadi lebih ringan dan bisa dipanggil kapan aja dari halaman manapun tanpa harus dibikin ulang.

Line 7: `val friends = listOf()`. Kita bikin variabel bernama `friends` yang tipenya adalah `List` (daftar berurutan). Semua data temanmu bakal dibungkus dan diurutkan di dalam sini.

Line 8: Data teman pertama. Kita manggil cetakan `Friend()` lalu ngisi datanya sesuai urutan: ID (1), Nama ("`Muhammad Lv Gunawuy`"), Tanggal nyelenehnya ("`32 Agustus 6969`"), cerita panjang soal heavenly demon, ID buat manggil fotonya (`R.drawable.lv`), dan link Instagram.

Line 9: Data teman kedua. Mengisi ID 2 untuk "`Fatih Bari Muar`" lengkap dengan tanggal "`29 Masehi 3802`", cerita soal inspirasi puisi di kamar mandi, dan ID fotonya (`R.drawable.fatih`).

Line 10: Data teman ketiga. Mengisi ID 3 untuk "`Meluyyyyyyyy`", cerita lucu soal aplikasi video chat, dan ID fotonya (`R.drawable.melon`).

Line 11: Data teman keempat. Mengisi ID 4 untuk "`neRvers.`", cerita soal lock in ibadah dan pentol di muka masjid, serta ID fotonya (`R.drawable.nafis`).

Line 12: Data teman kelima. Mengisi ID 5 untuk "`Rakha x Police`", cerita soal wawancara calon anggota HMTI FT ULM, dan ID fotonya (`R.drawable.rakha`).

Line 13: Kurung tutup) untuk mengakhiri batas isi dari listOf yang tadi dibuka di baris 7.

Line 14: Kurung kurawal tutup } untuk mengakhiri blok dari object FriendData.

Friend:

Line 1: Deklarasi package com.example.modul3xml.model. Ini menunjukkan kalau file ini disimpan di dalam folder model. Dalam dunia koding, folder model memang tempat khusus untuk menaruh rancangan atau cetakan dasar struktur data.

Line 3: Mengimpor androidx.annotation.DrawableRes. Ini adalah alat bantu penanda (anotasi) dari Android yang nanti dipakai di baris 10 untuk memastikan kevalidan file gambar.

Line 5: data class Friend(. Mendeklarasikan sebuah data class bernama Friend. Di Kotlin, data class itu ibarat formulir kosong yang memang diciptakan khusus untuk menyimpan data saja, tanpa ada rumus atau logika rumit di dalamnya. Tanda kurung buka (berarti kita mulai mendaftar isian formulirnya.

Line 6: val id: Int,. Isian pertama adalah id bertipe Int (angka bilangan bulat). Ini fungsinya seperti nomor antrean atau nomor identitas unik agar sistem gampang mencari data spesifik temanmu.

Line 7: val name: String,. Isian kedua, name bertipe String (teks). Digunakan untuk menyimpan nama teman.

Line 8: val date: String,. Isian ketiga, date bertipe String. Digunakan untuk menyimpan keterangan waktu atau tanggal nyeleneh dari setiap foto.

Line 9: val feature: String,. Isian keempat, feature bertipe String. Ini wadah untuk menampung teks cerita atau deskripsi panjang temanmu.

Line 10: @DrawableRes val photoResId: Int,. Isian kelima, photoResId bertipe Int. Nah, tambahan @DrawableRes di depannya adalah hasil dari import di baris 3 tadi. Fungsinya sebagai "satpam" yang memastikan kalau angka yang dimasukkan ke sini wajib berupa ID gambar resmi dari dalam folder Android (seperti R.drawable.fatih), bukan angka sembarangan.

Line 11: `val instagramUrl: String`. Isian keenam atau yang terakhir, `instagramUrl` bertipe `String`. Ini dipakai untuk menyimpan alamat link Instagram yang nanti akan disambungkan ke tombol agar bisa pindah aplikasi.

Line 12: `)`. Kurung tutup untuk mengakhiri daftar isian dari cetakan data class `Friend` tersebut.

Adapter.kt

Line 1: Deklarasi package `com.example.modul3xml.ui`. Ini menunjukkan kalau file `Adapter` ini disimpan di dalam folder `ui`.

Line 3 - 10: Blok import. Di sini kita memanggil alat-alat bantu penting seperti `Intent` (untuk pindah aplikasi), `RecyclerView` (untuk mesin daftarnya), `Glide` (untuk sihir gambar), `ViewBinding` (`ItemFriendBinding`) untuk membaca file XML, serta cetakan data `Friend`.

Line 12 - 16: Deklarasi kelas `FriendAdapter`. Kelas ini meminta dua kiriman data: `friends` (daftar teman-temanmu) dan `onDetailClick` (kabel penyambung klik). Kelas ini diatur menjadi keturunan dari `RecyclerView.Adapter` agar sah bertugas sebagai "penerjemah" data ke desain.

Line 18: Membuat inner class `FriendViewHolder`. Ini ibarat kelas kecil pembantu yang tugasnya memegang erat-erat satu bingkai desain item_friend.xml (yang diwakili oleh kata `binding`).

Line 20: Membuka fungsi `bind`. Di sinilah proses pemasangan data sungguhan dilakukan.

Line 22 - 24: Memasukkan teks. Data dari `friend.name`, `date`, dan `feature` ditempelkan ke komponen teks di layar yaitu `tvName`, `tvDate`, dan `tvFeature` (ID yang kamu buat di file desain XML).

Line 27 - 29: Memasukkan gambar menggunakan `Glide`. Alat ini akan mengambil ID foto (`photoResId`) lalu memasukkannya ke dalam kotak `ivPhoto` dengan sangat efisien agar aplikasi tidak lag (patah-patah) saat layar digeser dengan cepat.

Line 32 - 35: Memasang aksi klik untuk tombol `btnInstagram`. Di dalamnya kita menggunakan `Intent.ACTION_VIEW` untuk menerjemahkan teks URL Instagram menjadi

perintah untuk membuka browser atau aplikasi Instagram. Ini yang disebut dengan Intent Eksplisit di tugas modulmu.

Line 38 - 40: Memasang aksi klik untuk tombol btnDetail. Saat ditekan, tombol ini akan memicu fungsi navigasi onDetailClick sambil membawa bekal berupa nomor ID (friend.id) teman tersebut.

Line 41 - 42: Kurung tutup untuk mengakhiri isi fungsi bind dan kelas FriendViewHolder.

Line 45 - 48: Fungsi onCreateViewHolder. Ini adalah fungsi wajib pertama. Tugasnya adalah memompa (inflate) atau mencetak file desain item_friend.xml menjadi bingkai kosong yang nyata setiap kali layar HP membutuhkan kotak baru saat digulir ke bawah.

Line 51: Fungsi getItemCount. Fungsi wajib kedua yang bertugas menjawab pertanyaan sistem: "Ada berapa total teman di daftar ini?". Jawabannya diambil dari panjang total daftarnya (friends.size).

Line 54 - 56: Fungsi onBindViewHolder. Fungsi wajib ketiga ini bertugas menjodohkan bingkai kosong yang sudah dicetak tadi dengan data teman secara spesifik berdasarkan urutannya (position). Jadi teman nomor 1 pasti masuk ke kotak nomor 1.

Line 57: Kurung tutup terakhir untuk mengakhiri seluruh kode di dalam kelas FriendAdapter.

DetailFragment.kt

Line 1: Deklarasi package com.example.modul3xml.ui. Ini menunjukkan kalau file ini disimpan di dalam folder ui yang mengurus bagian antarmuka aplikasi.

Line 3 - 10: Kumpulan blok import. Bagian ini memanggil alat-alat bantu yang dibutuhkan, seperti Fragment (pengganti @Composable di XML), Glide buat ngatur gambar besar biar tidak lag, gudang FriendData, dan FragmentDetailBinding buat nyambungin kode Kotlin ini ke desain XML-nya.

Line 12: Deklarasi class DetailFragment. Kelas ini dibuat sebagai turunan dari Fragment(), yang artinya dia secara resmi bertugas sebagai satu halaman layar mandiri di arsitektur aplikasi ini.

Line 14 - 15: Menyiapkan variabel binding. Ini adalah cara wajib di sistem XML modern biar kita bisa memanggil ID dari desain `fragment_detail.xml` (kayak `tvDetailName` atau `ivDetailPhoto`) secara langsung, tanpa harus repot mengetik `findViewById` yang jadul.

Line 17 - 23: Fungsi `onCreateView`. Fungsi ini adalah langkah paling awal saat halaman mau dibuka. Tugasnya simpel: memompa (`inflate`) atau "mencetak" file desain XML (`fragment_detail.xml`) biar wujud visual bingkainya terbentuk di layar, lalu mengembalikannya (`return binding.root`).

Line 25 - 26: Fungsi `onViewCreated`. Fungsi ini dipanggil tepat setelah bingkai visualnya selesai dicetak. Di sinilah tempat kita mulai menyetel logika dan memasukkan data-data temanmu ke dalam bingkai tersebut.

Line 28 - 29: Kode untuk menangkap ID. Sistem akan membuka "paket" yang dikirim dari halaman list sebelumnya dengan mengambil nilai dari kunci `"friendId"`. Kalau kebetulan paketnya error atau kosong, nilai cadangannya disetel jadi 0.

Line 31 - 32: Kode pencarian data. Kita menyuruh sistem mencari ke dalam daftar `FriendData.friends`, lalu temukan satu data teman yang ID-nya sama persis dengan `friendId` yang baru saja ditangkap tadi.

Line 34: Pengecekan `if (friend != null)`. Ini adalah pintu pengaman untuk memastikan datanya beneran ketemu (tidak kosong) sebelum kita tempel-tempel ke layar.

Line 35 - 38: Memasukkan teks. Data nama, tanggal, dan cerita lengkap (feature) milik teman tersebut ditempelkan langsung ke dalam ID teks yang sudah kamu siapkan di file XML.

Line 40 - 44: Menampilkan foto menggunakan Glide. Alat ini akan mengambil ID gambar (`photoResId`), lalu memasukkannya ke kotak gambar `ivDetailPhoto`. Karena foto di detail ini ukurannya besar (350.dp), pakai Glide sangat membantu agar memori HP tidak langsung berat.

Line 45 - 46: Kurung kurawal tutup untuk mengakhiri blok pengecekan `if` dan blok fungsi `onViewCreated`.

Line 47 - 50: Fungsi `onDestroyView`. Ini adalah fungsi "bersih-bersih" yang wajib ada di setiap Fragment. Saat kamu memencet tombol back untuk keluar dari halaman detail ini, kita

harus mengosongkan memori dengan cara menyetel `_binding = null`. Tujuannya biar sisa-sisa tampilan layar ini tidak terus menumpuk dan membebani HP.

Line 51: Kurung tutup terakhir untuk mengakhiri keseluruhan class `DetailFragment`.

FriendListFragment

Line 1: Deklarasi package `com.example.modul3xml.ui`. Menandakan file ini disimpan di dalam folder `ui` karena tugas utamanya adalah mengurus tampilan layar.

Line 3 - 11: Blok import. Di sini kita memanggil semua komponen bantuan yang dibutuhkan, seperti `Fragment`, alat navigasi (`findNavController`), gudang data (`FriendData`), serta alat pembaca desain XML (`FragmentFriendListBinding`).

Line 13: Deklarasi class `FriendListFragment`. Kelas ini dibuat sebagai turunan dari `Fragment()`, yang artinya ia secara sah bertindak sebagai sebuah layar atau halaman mandiri di dalam aplikasi Android.

Line 16 - 17: Penyiapan `ViewBinding`. Ini adalah trik wajib di `Fragment` untuk menjembatani kode Kotlin dengan file desain XML-nya. Dengan variabel `binding` ini, kita bisa langsung memanggil ID komponen visual (seperti `RecyclerView`) tanpa perlu menggunakan perintah `findViewById` yang repot.

Line 20 - 26: Fungsi `onCreateView`. Fungsi ini bekerja seperti mesin cetak. Tugasnya adalah memompa (`inflate`) file XML `fragment_friend_list` agar wujud visual kerangkanya terbentuk di layar, lalu menampilkannya lewat `return binding.root`.

Line 29 - 31: Fungsi `onViewCreated`. Fungsi ini otomatis berjalan tepat setelah tampilan layar selesai dicetak. Di sinilah kita mulai mengatur aliran datanya.

Line 34: Mengambil seluruh data teman dari `FriendData.friends` dan memasukannya ke dalam variabel `friends` agar lebih ringkas dipanggil.

Line 36 - 39: Mengatur Rak Atas (`Featured Friends`). Kita membuat mesin penerjemah bernama `featuredAdapter` yang isinya hanya mengambil 5 teman pertama (`.take(5)`). Setelah itu, adapter ini langsung dipasang ke penampil daftarnya yaitu `rvFeaturedFriends`. Kita juga menyisipkan perintah navigasi kalau kartunya diklik.

Line 42 - 45: Mengatur Rak Bawah (All Friends). Kita membuat mesin penerjemah bernama `allAdapter`, tapi kali ini isinya adalah seluruh data friends tanpa dipotong. Adapter ini kemudian dipasangkan ke penampil daftar ke bawah yaitu `rvAllFriends`.

Line 49 - 50: Membuat fungsi `MapsToDetail` yang menerima kiriman angka ID teman (`friendId`). Fungsi ini adalah "kurir" pengantar perpindahan layar.

Line 51 - 53: Membuat bundle. Di Android XML, Bundle itu ibarat kardus paket. Kita memasukkan data ID teman ke dalam kardus ini menggunakan label "`friendId`".

Line 55: Memerintahkan sistem navigasi (`findNavController`) untuk segera berpindah menelusuri rute menuju layar Detail, sambil membawa kardus bundle yang berisi ID tadi.

Line 59 - 63: Fungsi `onDestroyView`. Ini adalah fungsi wajib untuk bersih-bersih. Ketika pengguna pindah layar atau menutup halaman ini, kita harus melepaskan kaitan `_binding = null`. Ini sangat penting bagi UX dan performa agar memori HP pengguna tidak cepat penuh (mencegah memory leak).

MainActivity.kt

Line 1: Deklarasi `package com.example.modul3xml`. Ini menandakan kalau file `MainActivity` ini letaknya ada di luar, alias langsung di folder utama proyek kita, tidak masuk ke sub-folder seperti `ui` atau `data`.

Line 3 - 5: Blok import. Di sini kita memanggil Bundle untuk menyimpan status layar saat aplikasi berjalan, `AppCompatActivity` sebagai komponen dasar layar Android, dan `ActivityMainBinding` untuk mengaktifkan fitur pembaca desain XML.

Line 7: Deklarasi `class MainActivity : AppCompatActivity()`. Kita membuat kelas utama bernama `MainActivity` yang mewarisi sifat-sifat dari `AppCompatActivity`. Karena ini adalah Activity, dia akan menjadi "tuan rumah" pertama saat aplikasimu di-klik di HP.

Line 9: `private lateinit var binding: ActivityMainBinding`. Kita membuat variabel bernama `binding` yang akan diisi belakangan (`lateinit`). Variabel ini tugasnya memegang semua ID komponen yang ada di dalam desain `activity_main.xml`.

Line 11: `override fun onCreate(savedInstanceState: Bundle?)`. Ini adalah fungsi bawaan Android yang otomatis langsung jalan paling pertama saat aplikasi mulai dibuat dan dibuka layarnya.

Line 12: `super.onCreate(savedInstanceState)`. Menjalankan tugas persiapan dasar dari sistem Android aslinya agar layarnya tidak error saat dibangun.

Line 14: Baris ini murni cuma komentar pengingat yang kamu tulis, tidak akan dibaca oleh mesin.

Line 15: `binding = ActivityMainBinding.inflate(layoutInflater)`. Ini adalah proses " meniup " atau mencetak file desain `activity_main.xml` agar wujud visualnya benar-benar terbentuk dan bisa dikendalikan lewat kode Kotlin.

Line 16: `setContentView(binding.root)`. Ini perintah pamungkasnya. Setelah desainnya dicetak, kita pasang dan tampilkan desain tersebut (`binding.root`) ke layar HP. Lewat kode inilah layar navigasi utamamu akhirnya bisa kelihatan.

Line 18: Kurung kurawal tutup untuk mengakhiri isi dari fungsi `onCreate`.

Line 19: Kurung kurawal tutup terakhir untuk mengakhiri keseluruhan class `MainActivity`.

MainActivity.xml

Line 1: `<?xml version="1.0" encoding="utf-8"?>`. Ini adalah baris wajib di setiap file XML. Fungsinya hanya memberi tahu sistem komputer bahwa dokumen ini ditulis menggunakan format bahasa XML versi 1.0 dengan standar karakter teks utf-8.

Line 2 - 4: Membuka wadah utama aplikasi menggunakan `ConstraintLayout` (layout yang sangat fleksibel karena pakai sistem tarik-menarik). Tiga baris `xmlns` (XML namespace) di bawahnya ibarat mendaftarkan kamus ke dalam file ini, agar ia paham saat kita mengetik perintah bawaan android, perintah khusus app, dan perintah bantuan tools.

Line 5 - 6: Mengatur ukuran wadah utamanya. Nilai `match_parent` pada lebar (`width`) dan tinggi (`height`) memerintahkan bingkai ini untuk melar mentok memenuhi seluruh ukuran layar HP dari ujung ke ujung.

Line 7: `tools:context=".MainActivity">`. Ini adalah alat bantu khusus desainer. Fungsinya untuk memberi tahu Android Studio bahwa desain XML ini adalah miliknya MainActivity, sehingga saat kita melihat preview desainnya, tampilannya lebih akurat. Tanda `>` di akhir digunakan untuk menutup tag pembuka `ConstraintLayout`.

Line 9 - 11: Membuka komponen bernama `FragmentContainerView`. Ini adalah "layar proyektor" atau kanvas kosong tempat halaman List dan Detail nanti ditayangkan bergantian. Kita beri dia nama ID `nav_host_fragment`, dan perintah `android:name` memastikan bahwa kanvas kosong ini khusus dikelola oleh mesin navigasi bawaan Jetpack.

Line 12 - 13: Mengatur ukuran si layar proyektor. Dalam aturan `ConstraintLayout`, nilai `0dp` berarti ukurannya akan merenggang mengikuti tarikan paku di sekelilingnya.

Line 14: `app:defaultNavHost="true"`. Perintah ini sangat penting untuk tombol Back (kembali) di HP. Ini membuat sistem tahu bahwa jika pengguna menekan tombol Back saat di halaman Detail, mereka harus dikembalikan ke halaman List, bukan malah dikeluarkan dari aplikasi.

Line 15 - 18: Ini adalah empat paku penarik (constraints). Kita memaku sisi bawah, kanan, kiri, dan atas dari layar proyektor ini langsung ke tepi layar utamanya (parent). Karena ditarik ke 4 arah mentok ke dinding layar, ukuran `0dp` tadi otomatis melar memenuhi seluruh area HP.

Line 19: `app:navGraph="@navigation/nav_graph" />`. Di sinilah kita memasukkan "kaset" peta jalannya. Perintah ini menyambungkan layar proyektor dengan file `nav_graph.xml` yang sudah mengatur alur perpindahan antar layarmu. Tanda `/>` di akhir digunakan untuk menutup komponen `FragmentContainerView` ini.

Line 20: `</androidx.constraintlayout.widget.ConstraintLayout>`. Tag penutup terakhir untuk membungkus dan mengakhiri wadah besar `ConstraintLayout` yang pertama kali kita buka di baris 2 tadi.

Fragment_detail.xml

Line 1: `<?xml version="1.0" encoding="utf-8"?>`. Baris standar yang memberi tahu sistem bahwa ini adalah file berekstensi XML dengan format teks utf-8.

Line 2 - 4: Membuka komponen ScrollView. Ini adalah wadah wajib agar halaman detail ini bisa digeser (di-scroll) ke bawah seandainya cerita temanmu sangat panjang. Tiga baris xmlns (XML namespace) mendaftarkan aturan penulisan android, app, dan tools.

Line 5 - 6: Mengatur ukuran ScrollView agar lebar (width) dan tingginya (height) melar memenuhi layar (match_parent).

Line 7: tools:context=".ui.DetailFragment">. Menandakan bahwa desain ini adalah pasangan dari file Kotlin DetailFragment.kt. Tanda > menutup pembukaan ScrollView.

Line 9: Membuka komponen LinearLayout. Karena ScrollView hanya boleh berisi satu anak saja, kita masukkan LinearLayout ini sebagai "keranjang" untuk menaruh gambar dan teks-teks di bawahnya.

Line 10 - 11: Lebar keranjang dibuat selebar layar (match_parent), tapi tingginya dibuat mengikuti isi di dalamnya saja (wrap_content).

Line 12: android:orientation="vertical". Ini aturan paling penting. Ia memaksa semua barang di dalam keranjang ini (foto, nama, tanggal, dll) disusun berbaris dari atas ke bawah.

Line 13: android:padding="16dp">. Memberi bantalan/jarak aman di sekeliling keranjang agar isinya tidak menempel ke tepi pinggir layar HP. Tanda > menutup pembukaan LinearLayout.

Line 15 - 20: Komponen ShapeableImageView untuk menampilkan foto profil besar.

- Line 16: Memberi ID ivDetailPhoto.
- Line 17 - 18: Lebarnya penuh (match_parent), tingginya dikunci di 350dp.
- Line 19: scaleType="centerCrop" memotong pinggiran gambar agar pas di kotak tanpa gepeng.

Line 20: app:shapeAppearanceOverlay="@style/RoundedImage" /> menerapkan gaya rahasia yang melengkungkan sudut gambar. Tanda /> menutup komponen gambar.

Line 22 - 29: Komponen TextView untuk nama. Diberi ID tvDetailName, jarak ke atas 16dp, ukuran teks besar 28sp, dan dicetak tebal (bold). Perintah tools:text="Nama Teman" hanya memunculkan teks palsu saat kamu melihat preview desain di Android Studio.

Line 31 - 39: Komponen TextView untuk tanggal. Diberi ID tvDetailDate, ukuran 16sp, dan diwarnai abu-abu gelap menggunakan kode hex #666666. Diberi margin atas dan bawah agar posisinya pas.

Line 41 - 46: Komponen TextView untuk label statis. Karena tidak perlu diubah dari Kotlin, komponen ini tidak perlu ID. Isinya langsung diisi android:text="Feature:" dengan ukuran 18sp dan cetak tebal.

Line 48 - 55: Komponen TextView untuk cerita lengkap. Diberi ID tvDetailFeature. Lebarnya dibuat mentok kiri-kanan (match_parent), ukuran teks 16sp, dan diberi lineSpacingExtra="8sp" agar jarak antar barisnya lebih renggang dan nyaman dibaca.

Line 57: </LinearLayout>. Tag penutup untuk mengakhiri keranjang LinearLayout.

Line 58: </ScrollView>. Tag penutup terakhir untuk mengakhiri wadah layar ScrollView.

Fragment_friend_list.xml

Line 1: <?xml version="1.0" encoding="utf-8"?>. Baris standar yang selalu ada di awal untuk menandakan ini adalah dokumen XML.

Line 2 - 7: Membuka wadah paling luar yaitu NestedScrollView. Kenapa tidak pakai LinearLayout biasa? Karena daftar temanmu mungkin sangat panjang, wadah ini memastikan seluruh area halaman bisa digeser (di-scroll) ke bawah. Baris tools:context di sini menyambungkan desain ini dengan file Kotlin FriendListFragment.

Line 9 - 13: Membuka komponen LinearLayout. Aturan di Android, wadah scroll hanya boleh memiliki satu anak langsung. Jadi kita butuh LinearLayout ini sebagai "keranjang" utamanya. Kita atur orientation="vertical" agar isi keranjang ini berbaris lurus dari atas ke bawah, dan paddingBottom="16dp" agar bagian paling bawah tidak terlalu menempel dengan dasar HP.

Line 15 - 21: Komponen TextView untuk judul rak pertama. Menampilkan teks "Featured Friends" dengan ukuran besar (20sp), dicetak tebal (bold), dan diberi jarak margin 16dp di sekelilingnya agar posisinya pas.

Line 23 - 30: Komponen RecyclerView untuk rak atas (geser menyamping).

Line 24: Diberi ID `rvFeaturedFriends` agar mudah dipanggil dari Kotlin.

Line 27: `clipToPadding="false"`. Ini trik rahasia desainer UI/UX! Ini membuat efek visual yang cantik saat kartu digeser, karena kartu tidak akan terpotong secara kaku oleh garis batas layar.

Line 28: `orientation="horizontal"`. Baris inilah yang mengubah sifat bawaan daftar dari yang tadinya ke bawah, menjadi berjejer menyamping.

Line 30: `app:layoutManager="..."` memberi tahu Android untuk menyusun isinya secara berurutan (linear).

Line 32 - 38: Komponen TextView untuk judul rak kedua. Sama seperti di atas, ini menampilkan tulisan "All Friends" sebagai pembatas sebelum masuk ke daftar yang utuh.

Line 40 - 45: Komponen RecyclerView untuk rak bawah (geser ke bawah).

Line 41: Diberi ID `rvAllFriends`.

Line 44: `android:nestedScrollingEnabled="false"`. halaman ini dibungkus oleh NestedScrollView (yang fungsinya buat scroll), kalau kita tidak mematikan fungsi scroll bawaan dari RecyclerView ini, akan nge-bug atau macet karena ada dua perintah scroll yang saling bentrok.

Line 47: `</LinearLayout>`. Tag penutup untuk mengakhiri keranjang penyusun vertikal.

Line 48: `</androidx.core.widget.NestedScrollView>`. Tag penutup paling akhir untuk mengakhiri wadah layar yang bisa digeser tersebut.

Item_friend.xml

Line 1: `<?xml version="1.0" encoding="utf-8"?>`. Baris pembuka standar yang menandakan dokumen ini menggunakan bahasa XML.

Line 2 - 10: Membuka komponen `MaterialCardView`. Ini adalah wadah paling luar yang berfungsi sebagai "bingkai kartu" untuk profil teman. Kita menggunakan komponen ini agar kartunya bisa diberi lengkungan di sudut (`cardCornerRadius="16dp"`) dan diberi efek bayangan tipis (`cardElevation="2dp"`) agar terlihat seperti 3D.

Line 12 - 17: Membuka `LinearLayout` pertama (Utama). Diatur dengan `orientation="horizontal"`. Ini adalah keranjang utama di dalam kartu yang membagi isi menjadi kiri dan kanan (foto di kiri, teks di kanan). Diberi jarak ke dalam (`padding="16dp"`) agar isinya tidak menempel ke garis tepi kartu.

Line 19 - 24: Komponen `ShapeableImageView`. Ini berfungsi untuk menampilkan foto profil teman. Ukurannya dikunci menjadi kotak pas `100dp x 100dp`. Aturan `centerCrop` memastikan foto tidak gepeng, dan `shapeAppearanceOverlay` memanggil gaya agar sudut foto ikut melengkung serasi dengan kartunya.

Line 26 - 31: Membuka `LinearLayout` kedua. Diatur dengan `orientation="vertical"`. Ini adalah keranjang khusus untuk menumpuk teks dan tombol dari atas ke bawah. Penggunaan `layout_weight="1"` di sini sangat penting; fungsinya menyuruh keranjang teks ini melar mengisi semua sisa ruang kosong di sebelah kanan foto profil.

Line 33 - 38: Membuka `LinearLayout` ketiga (Horizontal). Ini adalah keranjang kecil yang khusus dibuat hanya untuk menjejerkan baris Nama dan Tanggal secara bersebelahan.

Line 40 - 48: Komponen `TextView` untuk Nama. Diberi `layout_weight="1"` agar nama ini mengambil jatah ruang lebih banyak dan mendorong posisi teks tanggal mentok ke ujung kanan. Diberi batas maksimal 2 baris, dan jika namanya sangat panjang, akan dipotong halus dengan titik-titik di akhirnya (`ellipsize="end"`). (Catatan: Di kodemu, tag penutup teks nama dan pembuka teks tanggal menyambung di baris yang sama).

Line 48 - 54: Komponen `TextView` untuk Tanggal. Ukurannya lebih kecil (`10sp`) dan diberi jarak margin di sisi kiri (`marginStart`) agar tidak menempel dengan teks nama.

Line 55: `</LinearLayout>`. Tag penutup untuk mengakhiri keranjang kecil pembungkus baris nama dan tanggal.

Line 57 - 65: Komponen `TextView` untuk Deskripsi (`tvFeature`). Sama seperti nama, deskripsi ini dibatasi maksimal 2 baris agar kartu tidak menjadi terlalu panjang ke bawah, dan sisa teksnya disembunyikan menggunakan efek titik-titik (`ellipsize="end"`).

Line 67 - 72: Membuka `LinearLayout` keempat (`Horizontal`). Keranjang terakhir ini bertugas khusus untuk menjejerkan tombol-tombol di bagian paling bawah.

Line 74 - 80: Komponen `Button` untuk Instagram. Diberi ukuran lebar `0dp` dan `layout_weight="1"`.

Line 82 - 89: Komponen `Button` untuk Detail. Ini juga diberi lebar `0dp` dan `layout_weight="1"`. Karena kedua tombol ini menggunakan `weight` (bobot) yang sama yaitu 1, maka sistem akan secara otomatis membagi ruang keduanya persis rata 50:50.

Line 90 - 94: Kumpulan tag penutup. Baris-baris ini bertugas menutup semua keranjang `LinearLayout` yang bersarang tadi, dan diakhiri dengan penutup bingkai utama `MaterialCardView`.

Nav_graph.xml

Line 1: `<?xml version="1.0" encoding="utf-8"?>`. Ini adalah baris wajib pembuka yang memberi tahu komputer bahwa dokumen ini ditulis dalam bahasa XML.

Line 2 - 4: Membuka komponen `<navigation>`. Ini adalah lembaran peta utamanya. Tiga baris `xmlns` di bawahnya bertugas mendaftarkan kamus perintah agar sistem Android paham instruksi yang akan kita tulis.

Line 5: `android:id="@+id/nav_graph"`. Memberikan nama ID untuk peta jalan ini agar mudah dikenali oleh sistem.

Line 6: `app:startDestination="@id/friendListFragment">`. Baris ini sangat penting! Ini berfungsi sebagai "titik start". Kita menyuruh aplikasi untuk selalu membuka halaman `friendListFragment` paling pertama saat aplikasi baru dijalankan.

Line 8 - 12: Mendaftarkan titik lokasi (layar) pertama, yaitu <fragment>.

Line 9: Diberi ID friendListFragment.

Line 10: Disambungkan dengan file Kotlin FriendListFragment yang menjadi otak dari layar ini.

Line 11: Diberi label atau judul layar "Friend List".

Line 12: Disambungkan dengan file desain fragment_friend_list.xml agar kita bisa melihat preview visualnya di peta.

Line 14 - 16: Membangun jalan penghubung (<action>). Ini adalah rute perpindahan. Kita memberi ID untuk rute ini (action_friendListFragment_to_detailFragment), lalu mengarahkan tujuan akhir rute tersebut (destination) ke halaman detail. Rute inilah yang nanti akan kita panggil dari file Kotlin saat tombol ditekan.

Line 17: </fragment>. Tag penutup untuk mengakhiri pendaftaran layar pertama.

Line 19 - 23: Mendaftarkan titik lokasi (layar) kedua. Caranya sama persis dengan yang di atas. Diberi ID detailFragment, disambungkan ke Kotlin DetailFragment, diberi judul "Detail Friend", dan disambungkan ke file desain fragment_detail.xml.

Line 25 - 28: Membuka loket penerimaan paket (<argument>). Karena halaman detail ini butuh data dari halaman sebelumnya, kita harus mendata paket apa yang akan masuk.

Line 26: Paketnya diberi nama friendId.

Line 27: Tipe isi paketnya adalah integer (angka bilangan bulat).

Line 28: Jika ternyata ada error dan paketnya kosong, nilai cadangannya disetel menjadi 0 (defaultValue="0").

Line 29: </fragment>. Tag penutup untuk mengakhiri pendaftaran layar kedua.

Line 31: </navigation>. Tag penutup paling akhir untuk melipat dan menutup keseluruhan dokumen peta navigasi ini.

Pembahasan Compose

MainActivity.kt

Line 1: Merupakan deklarasi package file Kotlin yaitu `com.example.modul3compose`. Package ini menentukan lokasi file dan identitas unik dari aplikasi di dalam sistem Android.

Line 2: Baris kosong sebagai pemisah antara deklarasi package dan blok import.

Line 3 - 18: Mengimpor class dan library yang dibutuhkan oleh aplikasi:

- `android.os.Bundle`: digunakan saat activity dibuat untuk menyimpan status sebelumnya.
- `androidx.activity.ComponentActivity`: kelas dasar activity khusus untuk Jetpack Compose.
- `androidx.activity.compose.setContent`: fungsi untuk menetapkan konten UI menggunakan Composable, menggantikan `setContentView` di XML.
- `androidx.activity.enableEdgeToEdge`: fungsi untuk mengaktifkan tampilan layar penuh hingga ke area status bar dan navigation bar.
- `androidx.compose.foundation.layout.fillMaxSize`: modifier agar komponen UI mengisi seluruh ukuran layar.
- `androidx.compose.foundation.layout.padding`: modifier untuk memberikan jarak atau ruang kosong pada tampilan.
- `androidx.compose.material3.Scaffold`: komponen bawaan Material3 yang menyediakan struktur dasar layar modern.
- `androidx.compose.ui.Modifier`: class dasar untuk memodifikasi tampilan komponen Compose.
- `androidx.navigation.NavType`: digunakan untuk menentukan tipe data dari argumen navigasi.
- `androidx.navigation.compose.NavHost`: wadah utama yang menampilkan layar berbeda sesuai dengan rute navigasi.

- `androidx.navigation.compose.composable`: fungsi untuk mendefinisikan sebuah rute atau layar di dalam `NavHost`.
- `androidx.navigation.compose.rememberNavController`: fungsi untuk membuat dan menyimpan pengontrol navigasi (`NavController`).
- `androidx.navigation.navArgument`: digunakan untuk mendefinisikan argumen yang akan dikirim antar layar.
- `com.example.modul3compose.ui.DetailScreen`: mengimpor tampilan layar detail teman buatan sendiri.
- `com.example.modul3compose.ui.FriendListScreen`: mengimpor tampilan layar daftar teman buatan sendiri.
- `com.example.modul3compose.ui.theme.Modul3ComposeTheme`: mengimpor tema desain kustom untuk aplikasi.

Line 19: Baris kosong sebagai pemisah.

Line 20: Mendeklarasikan class `MainActivity` yang mewarisi `ComponentActivity()`. Class ini menjadi pintu masuk utama aplikasi yang menggunakan Jetpack Compose.

Line 21: Mendefinisikan *override* dari fungsi `onCreate()` yang dipanggil saat activity pertama kali dibuat oleh sistem.

Line 22: Memanggil `super.onCreate(savedInstanceState)` untuk menjalankan fungsi bawaan dari kelas induk agar siklus hidup aplikasi berjalan normal.

Line 23: Memanggil `enableEdgeToEdge()` agar konten aplikasi bisa tampil menembus area bar atas dan bawah *smartphone*.

Line 24: Memanggil `setContent { ... }`. Semua kode tampilan Compose akan diletakkan di dalam blok ini.

Line 25: Memanggil `Modul3ComposeTheme { ... }` untuk membungkus seluruh UI agar desain warna dan teksnya konsisten mengikuti tema aplikasi.

Line 26: Memanggil composable Scaffold dan memberinya Modifier.fillMaxSize() agar kerangkanya memenuhi layar penuh. Scaffold ini juga menghasilkan innerPadding yang akan dipakai oleh konten di dalamnya agar tidak tertutup sistem bar.

Line 27: Baris kosong.

Line 28: Mendeklarasikan variabel navController menggunakan fungsi rememberNavController(). Variabel ini bertugas sebagai supir yang mengarahkan perpindahan antar layar.

Line 29: Baris kosong.

Line 30 - 34: Memanggil NavHost sebagai peta navigasi aplikasi dengan pengaturan:

- Line 31: Menghubungkan NavHost dengan navController yang sudah dibuat.
- Line 32: Menetapkan list_screen sebagai layar pertama yang muncul saat aplikasi dibuka.
- Line 33: Memberikan padding bawaan dari Scaffold (innerPadding) agar konten tidak bertabrakan dengan pinggiran layar.

Line 35: Baris kosong.

Line 36 - 42: Mendefinisikan layar pertama menggunakan composable("list_screen"):

- Di dalamnya, memanggil tampilan FriendListScreen.
- Terdapat aksi onNavigateToDetail yang mana jika dipicu (diklik), akan menyuruh navController berpindah ke rute detail_screen/ dengan membawa data spesifik yaitu friendId.

Line 43: Baris kosong.

Line 44 - 47: Mendefinisikan layar kedua dengan rute dinamis yaitu detail_screen/{friendId}. Bagian arguments dideklarasikan untuk memberi tahu sistem bahwa layar ini menerima data bernama friendId dengan tipe data angka atau Integer (NavType.IntType).

Line 48: Membuat variabel id yang mengambil nilai argumen friendId dari tumpukan navigasi (backStackEntry). Jika nilainya kosong, maka akan diberi nilai *default* yaitu 0.

Line 49: Baris kosong.

Line 50: Memanggil tampilan `DetailScreen` dan memasukkan variabel `id` yang ditangkap sebelumnya agar layar bisa menampilkan data teman yang sesuai.

Line 51 - 58: Merupakan barisan kurung kurawal tutup `}` untuk mengakhiri blok `NavHost`, `Scaffold`, `Theme`, `setContent`, fungsi `onCreate`, dan terakhir menutup class `MainActivity`.

FriendData

Line 1: Merupakan deklarasi package file Kotlin yaitu `com.example.modul3compose.data`. Package ini menandakan bahwa file ini berada di dalam folder `data`, yang umumnya digunakan untuk memisahkan logika penyediaan data dari tampilan antarmuka (UI).

Line 2: Baris kosong sebagai pemisah antara deklarasi package dan blok import.

Line 3 - 4: Mengimpor file dan class yang dibutuhkan:

- `com.example.modul3compose.R`: digunakan untuk mengakses *resource* bawaan Android, seperti gambar yang tersimpan di dalam folder `drawable`.
- `com.example.modul3compose.model.Friend`: mengimpor *data class* `Friend` yang berfungsi sebagai cetakan atau model struktur data untuk setiap objek teman.

Line 5: Baris kosong sebagai pemisah.

Line 6: Mendeklarasikan object `FriendData`. Di dalam Kotlin, penggunaan kata kunci `object` menandakan pembuatan sebuah *Singleton*, artinya class ini hanya memiliki satu *instance* di seluruh siklus hidup aplikasi dan isinya bisa langsung diakses tanpa perlu membuat objek baru.

Line 7: Mendeklarasikan variabel `friends` yang bernilai `listOf(...)`. Ini berarti variabel `friends` menyimpan koleksi data berupa *List* yang bersifat *read-only* (tidak bisa ditambah/dikurangi setelah dibuat), di mana setiap elemen di dalamnya adalah instansiasi dari class `Friend`.

Line 8 - 15: Merupakan instansiasi objek `Friend` yang pertama di dalam *list*. Objek ini diisi dengan data *dummy* menggunakan parameter yang didefinisikan di data class `Friend`:

- `id = 1`: identitas unik dari data ini.

- `name`: menyimpan nama teman.
- `date`: menyimpan data tanggal atau keterangan waktu.
- `feature`: deskripsi panjang atau cerita lucu terkait teman tersebut.
- `photoResId = R.drawable.lv`: memanggil *resource* gambar bernama `lv` yang ada di dalam folder `res/drawable`.
- `instagramUrl`: tautan URL menuju profil Instagram.
- Line 15 ditutup dengan `)`, yang menandakan akhir dari objek pertama dan bersiap menerima objek kedua.

Line 16 - 23: Merupakan instansiasi objek `Friend` yang kedua, dengan `id = 2` atas nama "Fatih Bari Muar". Struktur pengisian propertinya (*name*, *date*, *feature*, gambar `R.drawable.fatih`, dan URL) sama persis dengan pola pada objek pertama.

Line 24 - 31: Merupakan instansiasi objek `Friend` yang ketiga, dengan `id = 3` atas nama "Meluyyyyyyy". Menggunakan gambar `R.drawable.melon`.

Line 32 - 39: Merupakan instansiasi objek `Friend` yang keempat, dengan `id = 4` atas nama "neRvers.". Menggunakan gambar `R.drawable.nafis`.

Line 40 - 47: Merupakan instansiasi objek `Friend` yang kelima sekaligus yang terakhir, dengan `id = 5` atas nama "Rakha x Police". Menggunakan gambar `R.drawable.rakha`. Pada baris ke-47, ditutup dengan `)` tanpa tanda koma di belakangnya karena ini adalah elemen terakhir di dalam `listOf`.

Line 48: Kurung tutup `)` untuk mengakhiri fungsi `listOf` pada variabel `friends`.

Line 49: Kurung kurawal tutup `}` untuk mengakhiri blok deklarasi object `FriendData`.

Friend

Line 1: Merupakan deklarasi package file Kotlin yaitu `com.example.modul3compose.model`. Penamaan model di ujung package menandakan bahwa file ini diletakkan khusus di dalam folder yang tugasnya hanya untuk mendefinisikan struktur data.

Line 2: Baris kosong sebagai pemisah.

Line 3: Mengimpor anotasi `androidx.annotation.DrawableRes`. Anotasi ini nantinya digunakan untuk memberi tahu sistem Android bahwa sebuah variabel angka (Integer) adalah sebuah ID gambar dari folder `drawable`.

Line 4: Baris kosong sebagai pemisah antara blok import dan deklarasi class.

Line 5: Mendeklarasikan data class `Friend`(. Di dalam Kotlin, data class adalah tipe class yang secara khusus dirancang hanya untuk menyimpan data. Dengan memakai data class, Kotlin otomatis membuatkan fungsi-fungsi berguna di belakang layar (seperti membandingkan atau menyalin data) tanpa perlu kita coding manual. Tanda kurung buka (menandakan awal dari daftar data apa saja yang dimiliki oleh class ini.

Line 6: Mendeklarasikan properti `id` dengan tipe data `Int` (angka bulat). Ini digunakan sebagai identitas unik atau nomor urut untuk setiap teman.

Line 7: Mendeklarasikan properti `name` dengan tipe data `String` (teks) untuk menyimpan nama teman.

Line 8: Mendeklarasikan properti `date` dengan tipe data `String` untuk menyimpan tanggal atau keterangan waktu terkait teman tersebut.

Line 9: Mendeklarasikan properti `feature` dengan tipe data `String` untuk menyimpan deskripsi, cerita, atau keterangan panjang tentang teman tersebut.

Line 10: Mendeklarasikan properti `photoResId` dengan tipe data `Int`. Terdapat anotasi `@DrawableRes` di depannya sebagai penanda khusus agar Android Studio bisa mengecek dan mencegah error jika kita salah memasukkan angka biasa yang bukan merujuk ke ID gambar.

Line 11: Mendeklarasikan properti `instagramUrl` dengan tipe data `String` untuk menyimpan tautan ke profil Instagram. Tidak ada tanda koma , di ujung baris ini karena ini adalah data terakhir di dalam class tersebut.

Line 12: Kurung tutup `)` untuk mengakhiri daftar properti dari data class `Friend`.

DetailScreen.kt

Line 1: Merupakan deklarasi package file Kotlin yaitu `com.example.modul3compose.ui`. Package ini menunjukkan bahwa file tersebut berada di dalam folder `ui`, yang bertugas mengatur tampilan antarmuka (User Interface).

Line 2: Baris kosong sebagai pemisah antara deklarasi package dan blok import.

Line 3 - 17: Blok import yang berisi semua fungsi Compose dan struktur data yang diperlukan. Termasuk di dalamnya komponen visual (Image, Text, Column), pengatur tata letak (Modifier), serta mengimpor gudang data yaitu `FriendData`.

Line 18: Baris kosong.

Line 19: Anotasi `@Composable` yang menandakan bahwa fungsi di bawahnya adalah komponen antarmuka yang dapat digambar (di-render) oleh sistem Jetpack Compose.

Line 20: Mendeklarasikan fungsi `DetailScreen()` dengan menerima dua parameter: `friendId` bertipe `Int` (sebagai kunci pencarian identitas teman), dan modifier untuk mengatur tata letak dengan nilai bawaan.

Line 21 - 22: Logika pencarian data. Membuat variabel `friend` yang bertugas masuk ke dalam `FriendData.friends` lalu mencari satu baris data (menggunakan fungsi `.find`) di mana nomor id teman cocok dengan nomor `friendId` yang dikirim ke halaman ini.

Line 23: Baris kosong.

Line 24: Membuka struktur pengkondisian `if (friend != null)`. Baris ini bertugas sebagai pengaman untuk memastikan bahwa blok kode tampilan di bawahnya hanya akan digambar jika data teman berhasil ditemukan (tidak kosong/null).

Line 25 - 30: Membuka komponen `Column` sebagai wadah utama yang menyusun elemen secara vertikal dari atas ke bawah.

- Line 27: `.fillMaxSize()`, memaksa wadah ini memenuhi seluruh layar HP.
- Line 28: `.padding(16.dp)`, memberikan jarak aman pada tepi layar agar tidak menempel di pinggir.
- Line 29: `.verticalScroll(rememberScrollState())`, menambahkan kemampuan gulir ke bawah agar pengguna bisa membaca deskripsi jika teksnya sangat panjang.

Line 31 - 39: Komponen Image yang bertugas menampilkan foto profil secara penuh.

- Line 32: `painterResource(id = friend.photoResId)`, mengambil gambar aslimu dari folder `drawable`.
- Line 34: `contentScale = ContentScale.Crop`, memotong gambar secara rapi agar pas memenuhi area tanpa membuat gambarnya gepeng.
- Line 36 - 38: Mengatur agar gambar memenuhi lebar layar (`.fillMaxWidth()`), tinggi dikunci di 350.dp, dan sudutnya dilengkungkan sebesar 16.dp menggunakan fungsi `.clip`.

Line 41: Komponen `Spacer` yang bertugas memberikan ruang kosong transparan (jarak) setinggi 16.dp antara gambar dan judul.

Line 43 - 49: Komponen `Text` untuk menampilkan nama lengkap teman (`friend.name`). Teks ini diatur menjadi sangat besar (28.sp), dicetak tebal (`FontWeight.Bold`), dan diberi jarak antar baris 32.sp.

Line 51 - 57: Komponen `Text` untuk menampilkan keterangan waktu/tanggal (`friend.date`). Teks ini berukuran 16.sp, menggunakan warna varian abu-abu dari tema bawaan (`onSurfaceVariant`), dan memiliki padding jarak ke atas dan ke bawah.

Line 59 - 64: Komponen `Text` statis yang hanya menampilkan tulisan "Feature:". Ukurannya diatur 18.sp dan dicetak tebal sebagai penanda sebelum masuk ke deskripsi utama.

Line 66: Komponen `Spacer` kecil untuk memberi jarak 4.dp agar tulisan "Feature:" dan isinya tidak terlalu berdempetan.

Line 68 - 73: Komponen `Text` terakhir untuk menampilkan isi paragraf panjang (`friend.feature`) dengan ukuran standar 16.sp dan jarak antar baris 24.sp (`lineHeight`) agar nyaman dibaca.

Line 74: Menutup blok fungsi komponen `Column`.

Line 75: `} else {`, membuka kondisi balikan jika ID yang dicari ternyata tidak ada di dalam `FriendData`.

Line 76 - 79: Komponen pengaman/fallback. Membuka wadah Box yang memenuhi seluruh layar dengan penyalarsan di tengah persis (Alignment.Center). Di dalamnya terdapat Text sederhana yang akan memberitahu pengguna: "Data teman tidak ditemukan".

Line 80: Menutup blok kondisi else.

Line 81: Menutup keseluruhan fungsi @Composable DetailScreen.

FriendItem.kt

Line 1: Deklarasi package com.example.modul3compose.ui. Ini menunjukkan struktur map (direktori) tempat file ini disimpan, yaitu di dalam folder ui yang khusus mengurus tampilan antarmuka.

Line 3 - 13: Kumpulan blok import. Bagian ini memanggil semua "alat bantu" yang dibutuhkan dari Jetpack Compose, seperti pengatur tata letak (layout), komponen daftar gulir yang ramah memori (LazyColumn, LazyRow), hingga mengimpor gudang data FriendData milikmu.

Line 15: Anotasi @Composable. Ini adalah penanda wajib bagi mesin Android bahwa fungsi di bawahnya bukanlah fungsi logika matematika biasa, melainkan sebuah komponen visual yang akan digambar ke layar HP.

Line 16 - 19: Deklarasi fungsi utama FriendListScreen. Fungsi ini membutuhkan parameter onNavigateToDetail: (Int) -> Unit. Parameter ini bertindak sebagai "kabel penyambung"; ia menunggu layar ini mengirimkan angka ID teman, lalu kabel tersebut akan meneruskannya ke MainActivity untuk melakukan perpindahan halaman.

Line 21: Pembuatan variabel friends. Kode ini mengambil seluruh isi data teman dari FriendData.friends dan menyimpannya ke dalam satu wadah pendek agar kita tidak perlu mengetik panjang-panjang saat memanggilnya nanti.

Line 23 - 26: Membuka komponen LazyColumn. Berbeda dengan Column biasa, LazyColumn sangat hemat memori karena ia hanya me-render kartu yang sedang terlihat di layar. Kita beri modifier = modifier.fillMaxSize() agar area yang bisa digulir (scrollable area) membentang memenuhi seluruh layar HP.

Line 27 - 34: Blok item pertama. Di dalam LazyColumn, jika kita hanya ingin memasukkan satu elemen tunggal (bukan sebuah daftar), kita wajib membungkusnya dengan item { }. Di sini kita memasukkan Text untuk judul "Featured Friends" agar judul ini ikut tergeser saat layar di-scroll ke bawah.

Line 36 - 40: Blok item kedua yang berisi LazyRow. Ini adalah wadah untuk membuat daftar gulir menyamping (horizontal scroll). Kita menambahkan contentPadding 16.dp agar di ujung kiri dan kanan layar ada ruang bernapas (tidak menabrak bezel HP), serta Arrangement.spacedBy(16.dp) agar ada jarak rapi antar kartu.

Line 42: Perintah items(friends.take(5)). Fungsi .take(5) secara matematis memotong daftar data agar hanya mengambil 5 urutan teratas. Ini sangat cocok untuk membuat fitur Highlight atau "Teman Unggulan".

Line 43 - 48: Komponen FriendItem yang dibungkus oleh Box. Penggunaan Box dengan ukuran pasti (width = 350.dp) ini adalah trik desain UX. Tujuannya agar kartu tidak melar memenuhi 100% layar. Dengan begitu, kartu di sebelahnya akan sedikit "mengintip", memberi isyarat visual kepada pengguna bahwa mereka bisa menggeser layar ke samping. Di sini juga disisipkan aksi onDetailClick untuk mengirimkan friend.id ke parameter navigasi.

Line 51 - 52: Kurung kurawal untuk menutup elemen LazyRow dan penutup blok item kedua.

Line 54 - 61: Blok item ketiga. Sama fungsinya seperti yang pertama, yaitu sebagai penyekat atau pemisah antar bagian dengan menampilkan teks judul "All Friends".

Line 63 - 68: Perintah items(friends). Di sini kita mencetak seluruh data tanpa memotongnya. Karena ia diletakkan langsung di dalam LazyColumn (tanpa LazyRow), kartu-kartu ini secara otomatis akan berbaris vertikal ke bawah mengikuti sifat dasar LazyColumn.

Line 70 - 71: Kurung kurawal terakhir untuk menutup struktur LazyColumn dan fungsi @Composable secara keseluruhan.

SOAL 2

Mengapa RecyclerView masih digunakan, padahal RecyclerView memiliki kode yang panjang dan bersifat boiler-plate, dibandingkan LazyColumn dengan kode yang lebih singkat?

A. Penjelasan

RecyclerView masih banyak digunakan karena ia lebih stabil dan efisien dalam segi performa, terutama untuk daftar data besar dan kompleks, meskipun kodenya lebih panjang dan bersifat boilerplate dibanding LazyColumn. Alasan terkuat mengapa RecyclerView masih mendominasi adalah karena jutaan aplikasi Android yang sudah berjalan dibangun di atas sistem View berbasis XML. Migrasi ke Compose direkomendasikan dilakukan secara bertahap (*incremental migration*), di mana Compose dan View berdampingan dalam satu codebase hingga aplikasi sepenuhnya berpindah ke Compose. Dalam hal performa scrolling mentah (*raw scrolling performance*), RecyclerView memiliki sedikit keunggulan dibandingkan LazyColumn berkat optimasi yang sudah matang dan kontrol level bawah yang lebih dalam. [1]

Namun, performa LazyColumn sangat baik dan terus meningkat di setiap rilis Compose. Secara teknis, perbedaannya terletak pada mekanisme *recycling*-nya. LazyColumn tidak mendaur ulang *children* yang sudah dibuat seperti RecyclerView ia justru membuat Composable baru setiap kali item baru muncul saat di-scroll, namun ini tetap efisien karena membuat Composable jauh lebih murah secara sumber daya dibandingkan membuat Android View baru. [1]

RecyclerView dikenal karena optimasi performa seperti *view recycling* dan *efficient scrolling*, serta menawarkan lebih banyak fleksibilitas dalam mengkustomisasi layout item, menangani berbagai tipe item yang berbeda, dan mengintegrasikan animasi kompleks. LazyRow/LazyColumn, meskipun lebih efisien dalam penulisan kode, mungkin memiliki keterbatasan tertentu dalam opsi kustomisasi. [2]

RecyclerView telah hadir sejak 2014. Ini berarti selama lebih dari satu dekade, ekosistemnya telah berkembang sangat luas. RecyclerView adalah komponen yang sudah matang dan banyak digunakan dalam pengembangan Android, sehingga lebih mudah menemukan sumber daya dan dukungan komunitas. LazyRow/LazyColumn, yang relatif

baru dalam Jetpack Compose, mungkin mengharuskan developer untuk terlebih dahulu membiasakan diri dengan framework Compose. Boilerplate memang nyata adanya implementasi RecyclerView membutuhkan banyak kode boilerplate dan berulang, termasuk pengaturan adapter, ViewHolder, dan logika recycling yang kompleks. Namun, boilerplate itu sendiri bukanlah alasan yang cukup untuk beralih jika konteks proyeknya tidak mendukung. Seperti analogi seorang arsitek yang memilih material bangunan: batu bata lebih sulit dipasang dari panel prefabrikasi, tapi tidak berarti batu bata tidak digunakan lagi.[3]

Untuk kode yang sudah ada atau layout yang sangat dikustomisasi, RecyclerView mungkin masih relevan, sedangkan LazyColumn adalah pilihan masa depan untuk proyek baru yang menggunakan Jetpack Compose. [3]

RecyclerView masih banyak digunakan karena sebagian besar aplikasi Android yang sudah berjalan dibangun di atas sistem View berbasis XML, sehingga mengganti seluruhnya ke LazyColumn membutuhkan biaya, waktu, dan risiko yang besar dan Google sendiri merekomendasikan migrasi dilakukan secara bertahap. Selain itu, meskipun kode RecyclerView lebih panjang, ia menawarkan kontrol dan fleksibilitas yang lebih dalam untuk kasus-kasus kompleks seperti banyak tipe item, animasi kustom, dan dekorasi daftar, serta didukung oleh ekosistem, dokumentasi, dan komunitas yang jauh lebih matang dibanding Jetpack Compose yang baru stabil pada 2021. Singkatnya, boilerplate bukan alasan cukup untuk beralih jika konteks proyek, keahlian tim, dan arsitektur yang ada belum mendukung RecyclerView dipilih bukan karena lebih mudah ditulis, melainkan karena lebih aman dan terbukti untuk situasi tertentu. [1][3]

REFERENSI

- Nugroho, B. P., Susanto, A., & Wibowo, C. (2024). Analisis Perbandingan Performa RecyclerView dan LazyColumn dalam Menampilkan Koleksi Data pada Aplikasi Android. *Jurnal Pengembangan Teknologi Informasi dan Ilmu Komputer*, 8(2), 112–120.
- Pratama, A., & Kurniawan, R. (2024). A Study on XML vs. Compose for Dynamic UI Rendering in Android Applications. *International Journal of Software Engineering and Applications*, 15(4), 45–58.

TAUTAN GIT

<https://github.com/ZulFath1/PraktikumMobile.git>